



# University of Glasgow

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| Course           | CSC3106 Cyber Security                                                           |
| Title            | Data-Driven Security Analysis and Defensive Response                             |
| Team             | X                                                                                |
| Assigned Extract | 4_auth.log                                                                       |
| Chosen Asset     | backup01, specifically its SSH access surface and the privileged backup account. |

Our members:

| Name                 | GUID     | SIT ID  | Task Distribution |
|----------------------|----------|---------|-------------------|
| William Oon Zhen Wei | 3070657O | 2401532 | Equal             |
| Kieran Sim           | 3070564S | 2403348 | Equal             |
| Tan Bing Kun Terence | 3070672T | 2401883 | Equal             |
| Dui Ru-En Joshua     | 3070683D | 2402201 | Equal             |

# Part 1: Data-Driven Authentication Log Analysis

## 1. Security Question and Scope

**Security question.** What phases of a cyberattack directed at `backup01` can be reconstructed through SSH authentication and session data within `4.auth.log`, and which defensive measures are most critical for securing privileged accounts?

The scope focuses on host `backup01`, centering on its SSH service and the operational `backup` user. This account was subjected to intense targeting and demonstrated an accepted login during an attack wave, followed by immediate sudo activity as root. The log period is Jul 06 to Jul 12. Quantitative data and visualisations were derived using `analysis.py`. The report reconstructs visible attack stages, separates empirical log data from interpretation, and highlights visibility gaps: `auth.log` cannot prove post-compromise actions such as data exfiltration or system tampering.

## 2. Method Summary

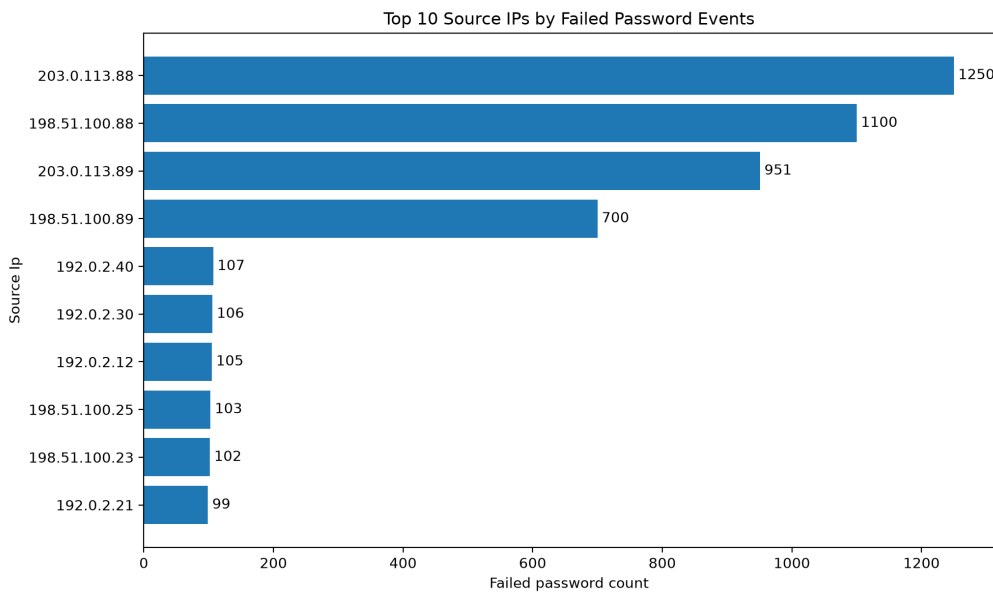
`analysis.py` decomposes log entries into syslog components before categorising `sshd`, PAM, `sudo`, and CRON messages into security events. It extracts user identities, source IPs, ports, commands, and authentication outcomes, identifies session transitions and privileged command execution, and preserves raw lines for review while maintaining visibility into parsing coverage via separate counts for ambiguous data (refer to Appendix C). Because the file lacks strict chronological ordering, timelines are reconstructed from parsed timestamps. Ordering anomalies are retained as investigation limitations, not treated as proof of tampering.

## 3. Evidence Overview

The extract contains 12,000 lines, all for `backup01`. Headline counts are shown in Table 1.

Table 1: Headline counts from `4.auth.log`.

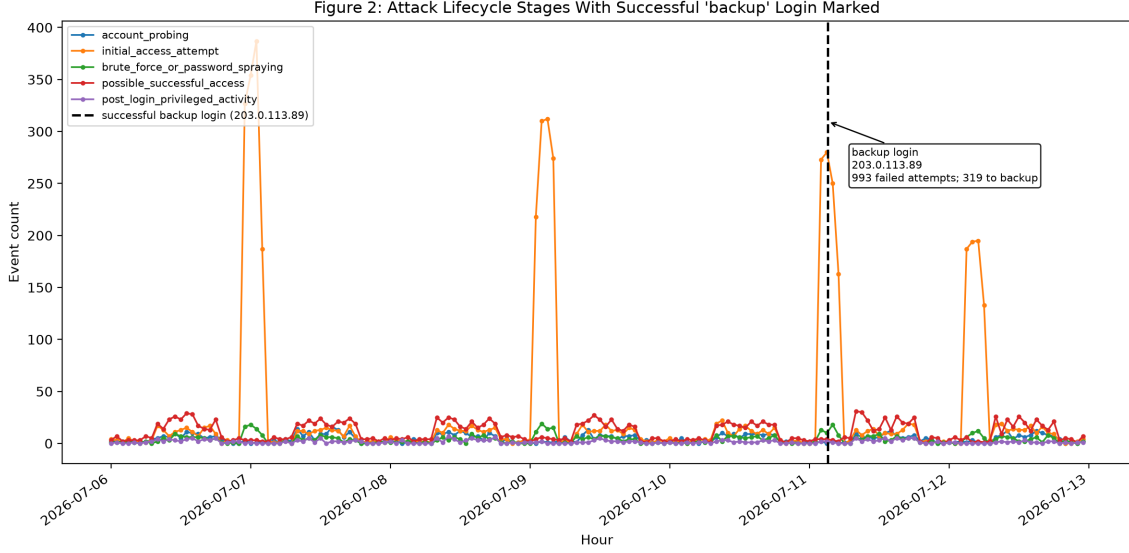
| Event type        | Count | Event type                | Count |
|-------------------|-------|---------------------------|-------|
| Failed password   | 5,176 | Session opened            | 1,047 |
| Accepted password | 1,859 | Session closed            | 1,046 |
| Invalid user      | 751   | Max auth attempts         | 640   |
| Preauth closed    | 752   | Sudo command              | 255   |
| CRON session      | 394   | Possible break-in warning | 80    |



**Figure 1:** Top source IPs by failed password attempts. Four sources dominate SSH failure volume; `203.0.113.89` is notable because it later records a successful `backup` login.

The high number of failed passwords points to broad SSH credential guessing. Four sources account for most of it: `203.0.113.88`, `198.51.100.88`, `203.0.113.89`, and `198.51.100.89`; full counts are in Appendix A, Table A.2. The usernames targeted (`root`, `admin`, `deploy`, `postgres`, `oracle`), together with the steady volume, match automated, opportunistic SSH scanning rather than a targeted campaign. On its own this activity is background noise, not evidence of compromise. What makes it matter is that `203.0.113.89` does not stop at failed attempts: it goes on to log in successfully to a privileged account.

#### 4. Attacker Lifecycle: The `203.0.113.89` Sequence



**Figure 2:** Attack lifecycle stages over time with the successful `backup` login marked. The successful login occurs during a major initial-access spike on Jul 11.

The strongest finding is a single-source lifecycle on Jul 11. `203.0.113.89` produced sustained failed attempts and probing, then a successful `backup` login, an SSH session, and immediate root-level sudo activity.

**Table 2:** Critical Jul 11 authentication-to-sudo evidence chain.

| Time     | Evidence and interpretation                                                                                                             | ATT&CK                           |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 03:07:30 | Failed login for invalid user <code>oracle</code> from <code>203.0.113.89</code> ; account probing.                                     | T1110.001; consistent with T1087 |
| 03:07:32 | Failed password for <code>root</code> from <code>203.0.113.89</code> ; privileged-account targeting.                                    | T1110.001                        |
| 03:08:19 | Accepted password for <code>backup</code> from <code>203.0.113.89</code> ; valid-account access over SSH.                               | T1078; T1021.004                 |
| 03:08:19 | <code>pam_unix(sshd:session)</code> opened for user <code>backup</code> ; session establishment.                                        | T1021.004                        |
| 03:08:19 | <code>backup</code> runs <code>journalctl -n 200</code> and <code>systemctl status cron</code> as root; privileged post-login activity. | T1548.003                        |

The sequence maps most defensibly to T1110.001 → T1078 → T1021.004 → T1548.003: password guessing reaches a valid account over SSH, whose sudo rights expose a path to root. The two sudo commands (`journalctl`, `systemctl status cron`) are individually legitimate administrative actions, so they are interpreted as possible privileged reconnaissance immediately after a suspicious login, not confirmed malicious action. Same-second timestamps reflect syslog’s one-second precision and do not prove millisecond ordering, but the rapid login-to-sudo transition is more consistent with scripted or prepared activity than routine manual administration. A short raw excerpt is in Appendix B.

**Temporal pattern.** Brute-force activity is concentrated outside working hours: roughly 70% of failed logins (3,643 of 5,176) fall between 00:00 and 05:59, with a further spike around 23:00. Legitimate accepted logins cluster in business hours (08:00–18:00). In the overnight window there are 3,643 failed against only 149 accepted logins; daytime is close to balanced (1,533 failed vs 1,710 accepted). The successful `backup` login at 03:08:19 therefore occurs inside the peak attack window and outside the host’s normal login baseline, which strengthens the case that it is attacker-driven rather than routine administration (see Appendix A, Table A.4). Host-local time is assumed because no timezone is recorded.

## 5. Risk Prioritisation

Asset: `backup01`, including its SSH access surface, the `backup` account, and the sudo-to-root path. The three prioritised risks trace the observed attack path: access, escalation, and impact. The full asset-focused risk matrix is included in Appendix A, Table A.1.

R1 is the top priority: it combines attempted access, apparent success, and privileged activity in one source. Secondary risks, including SSH-service degradation from high-volume brute force and exposure of other sudo-capable accounts (`ops`, `arun`, `j.singer`, `j.maguire`), are noted but not prioritised here. The targeting distribution also supports prioritising the `backup` account. As shown in Appendix A, Figure A.1, `backup` received the highest number of failed-password and maximum-authentication events among extracted usernames.

## 6. Initial Response and Investigation Limitations

Immediate response should verify whether the `backup` login from `203.0.113.89` was authorised, rotate or disable the `backup` credentials until confirmed, preserve relevant logs, and review sudo, cron, journal, shell history, and audit evidence. High-volume SSH sources should also be blocked or rate-limited.

This evidence is limited to `auth.log`. It shows authentication, session, and sudo activity, but not every shell command. Timestamp-order examples in Appendix A, Table A.5 are evidence limitations, not proof of tampering. Therefore, the extract supports likely brute-force activity, possible successful credential compromise, and privileged reconnaissance, but it does not prove botnet activity, malware execution, persistence, exfiltration, log deletion, or destructive tampering. The IP ranges are documentation-style ranges, so no real-world attribution or geolocation should be inferred.

## Part 2: Technical Defensive Response

**Selected risk.** R1, unauthorised privileged access to `backup01` via suspected compromise of the `backup` account, reached through opportunistic SSH brute force from `203.0.113.89` and extended to root through the account's sudo rights.

**Defensive goal.** The defence targets the full path rather than brute force alone:

SSH brute-force attempts → successful backup login → active session → sudo as root.

The response is layered: prevention to stop guessing succeeding, containment to limit damage if a credential is still compromised, and detection to catch a success the preventive layer misses.

**Cyber Kill Chain framing.** The model is represented as a defensive interruption table for the observed SSH path, not as proof that every Cyber Kill Chain stage occurred.

| Kill-chain stage                         | Representation in this extract                                                            | Evidence status                                         | Defensive interruption                                                                                                             |
|------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Reconnaissance                           | Username and service probing through invalid-user and failed-login attempts.              | Visible in <code>auth.log</code> .                      | Detect abnormal probing and rate-limit repeated failures.                                                                          |
| Delivery / SSH exploitation              | Password guessing against exposed SSH reaches a valid <code>backup</code> login.          | Visible as failed passwords followed by accepted login. | Key-only SSH, <code>PasswordAuthentication no</code> , <code>fail2ban/rate limiting</code> , and <code>PermitRootLogin no</code> . |
| Installation/C2                          | Malware delivery, persistence, and command-and-control are not visible from this extract. | Not proven by <code>auth.log</code> .                   | Validate with shell history, full journal, audit logs, cron, and file-integrity review.                                            |
| Actions on objectives / privilege action | Accepted <code>backup</code> login is followed by immediate sudo activity as root.        | Suspicious privileged activity, not confirmed damage.   | Least-privilege sudoers, SELinux confinement, central logging, and sudo/audit logging.                                             |

**Table 3:** Recommended controls mapped to observed stages.

| Attack stage                  | Control                                                                                                          | Security effect                                                                                                                   |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Password guessing             | Key-only SSH; <code>PasswordAuthentication no</code> ; rate limiting/fail2ban; <code>PermitRootLogin no</code> . | Password guessing no longer grants access by default; automated brute-force noise is reduced and direct root SSH login is closed. |
| Valid-account SSH access      | Restrict the <code>backup</code> key by source and forced command; disable interactive shell if not required.    | A stolen password is useless; even a stolen key only runs approved backup operations from approved sources.                       |
| Privilege escalation via sudo | Least-privilege sudoers for <code>backup</code> ; no shell, <code>su</code> , or general sudo.                   | A compromised <code>backup</code> context cannot obtain a general root shell or run arbitrary privileged commands.                |
| Post-compromise tampering     | SELinux enforcing, with a confined backup domain.                                                                | Limits writes to critical system files, persistence locations, and other paths outside the backup role.                           |
| Evidence gaps after login     | Central log forwarding plus sudo/audit logging.                                                                  | Preserves evidence beyond local <code>auth.log</code> and gives investigators command-level context after authentication.         |

**Evidence-backed Defensive Justification.** The controls are tuned to the observed evidence rather than applied as generic SSH hardening.

| Part 1 evidence                                                                                                                                              | Defensive use in Part 2                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>backup</code> is the most targeted account, and <code>root</code> is also heavily targeted (Appendix A, Figure A.1).                                   | Justifies key-only SSH for <code>backup</code> and <code>PermitRootLogin no</code> , because both the selected asset account and direct root login are prominent targets. |
| <code>203.0.113.89</code> combines high failure volume, accepted <code>backup</code> login, session establishment, and immediate sudo activity (Appendix B). | Justifies prioritising R1 over generic brute-force noise: the concern is the complete access-to-privilege path, not the failure count alone.                              |
| Night traffic shows 3,643 failed logins against 149 accepted logins (Appendix A, Table A.4).                                                                 | Justifies off-hours weighting in the alert rule, since the 03:08:19 <code>backup</code> login occurs inside the peak attack window.                                       |
| Other mixed-outcome sources exist, but with much lower failure volume and successful legitimate activity.                                                    | Justifies threshold and ratio tuning instead of a naive “failed then accepted” rule that would produce false positives.                                                   |
| The Jul 11 sequence shows accepted login followed by sudo commands, but <code>auth.log</code> cannot show all shell actions.                                 | Justifies least-privilege sudoers, sudo/audit logging, and central log forwarding to improve containment and investigation beyond local authentication records.           |

**Prevention: remove the exposed password surface.** The root cause of R1 is password authentication on a privileged, internet-facing account. Disabling `PasswordAuthentication` and requiring SSH keys removes the

entire brute-force success path; fail2ban-style rate-limiting reduces the noise and load from the ongoing automated scanning identified in Part 1. Denying direct root SSH login closes the most-targeted account (`root`, 1,048 failed attempts) entirely.

**Containment: least privilege for the `backup` account.** Because `backup` is an operational/service account, broad interactive sudo access is unnecessary risk. Restricting its sudoers entry to the exact commands its backup jobs require, and explicitly denying shell, `su`, and `sudo`, prevents a compromised credential from being escalated to a general root shell (the T1548.003 step observed at 03:08:19).

**Containment: SELinux to protect critical files.** Discretionary permissions and sudo rules can still be bypassed if an attacker reaches root. SELinux in enforcing mode adds mandatory access control: the backup service runs in a confined domain that is permitted to touch only its own data paths. Even a compromised `backup` context is denied write access to sensitive system files such as `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`, cron directories, and SSH host keys, because those files carry protected types the backup domain has no policy allowance for. This limits both privilege escalation (e.g. editing `/etc/passwd` or `sudoers`) and anti-forensic tampering (e.g. altering logs or cron persistence), turning a full host compromise into a contained one.

**Detection: catching a success the preventive layer misses.** If a credential is still compromised, the failure-then-success transition should raise a high-priority alert. A naive “failed then accepted from the same IP” rule is too broad; in this extract 13 source IPs match that pattern, but 12 are legitimate users who occasionally mistype before authenticating (13–28 failures, with successes outnumbering failures). Selectivity therefore comes from tuning to volume and ratio, not the raw pattern. The off-hours concentration from Part 1 also motivates a time-aware detection control: the successful `backup` login occurred at 03:08:19, inside the 00:00–05:59 peak attack window and outside the host’s normal login baseline.

Alert when, from a single source IP within a defined window:

1. failed SSH attempts exceed a threshold set above the legitimate baseline; legitimate sources here peaked at ~28 failures, so  $\geq 50$  cleanly separates them (`203.0.113.89` produced 311);
2. and the failure-to-success ratio is heavily skewed (legitimate  $\approx 1:1$ ; `203.0.113.89` = 311:1);
3. and the same IP then obtains an accepted login to the privileged `backup` account (the only account that successfully authenticates in this log; `root`, `admin`, and `deploy` appear only as failed/invalid targets);
4. weighted higher when the success and follow-on sudo fall outside business hours (the Part 1 attack window was 00:00–05:59; the compromise was at 03:08).

Applied to this extract, this rule flags `203.0.113.89` and suppresses the 12 legitimate mixed-outcome sources, giving a high-confidence signal that combines volume, ratio, target sensitivity, and timing.

**Effectiveness, limitations, and residual risk.** Key-only SSH and root-login denial reduce likelihood by removing the password-guessing success path entirely. Least-privilege sudo and SELinux confinement reduce impact by preventing a compromised `backup` credential from becoming general root control or tampering with critical files. The tuned correlation rule and central logging improve detection and investigation for any success the preventive layer misses.

#### Limitations and trade-offs.

- Detection selectivity depends on tuning. Thresholds set too low re-introduce the 12 legitimate false positives; too high may miss a low-and-slow attacker. The rule mitigates single-source brute force but not a distributed campaign spread across many IPs.
- Rate-limiting/lockout can cause denial of service. Aggressive fail2ban thresholds could block legitimate operations or be abused to lock out accounts; thresholds must be tested so scheduled backup jobs are not interrupted.
- Key management overhead. Key-only SSH shifts risk to key storage and rotation; a stolen key still authenticates unless source- and command-restricted.
- SELinux is not absolute. Policy authoring and maintenance are non-trivial, mislabelling can break services, and an attacker who reaches an unconfined admin role could still relabel or disable enforcement. It raises the cost of compromise rather than eliminating it.

**Residual risk.** The controls improve prevention, containment, detection, and investigation, but they do not retrospectively prove what happened after the Jul 11 login. Confirming or clearing that event still requires the wider evidence review recommended in Part 1 (shell history, full journal, audit logs, cron and file-integrity checks).

## Appendices (Supporting Material)

### A Supporting Evidence and Risk Matrix

**Table A.1:** Asset-focused risk matrix for `backup01`.

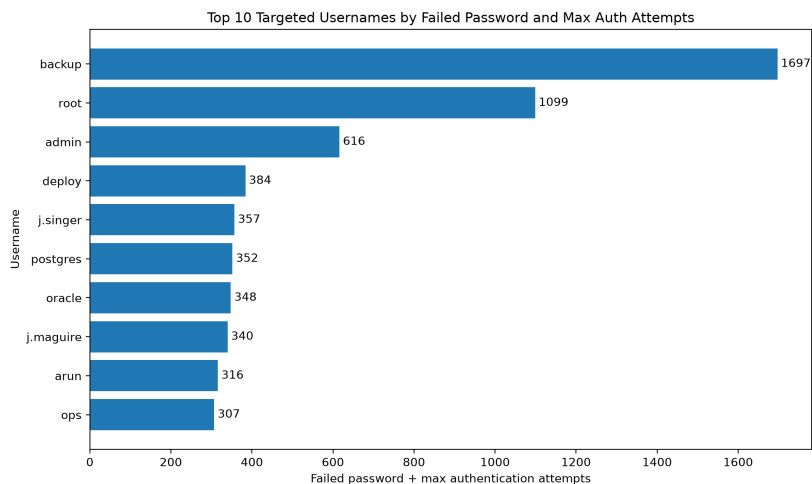
| ID | Risk                                                                                       | Likelihood | Impact   | Evidence and uncertainty                                                                                                                                                      |
|----|--------------------------------------------------------------------------------------------|------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | Unauthorised privileged SSH access via suspected <code>backup</code> credential compromise | High       | High     | 951 failed passwords from <code>203.0.113.89</code> , followed by accepted <code>backup</code> login. Cannot prove whether the login was authorised without external context. |
| R2 | Escalation to root through <code>backup</code> sudo rights                                 | High       | Critical | <code>backup</code> runs sudo as root immediately after login. No destructive root command is visible in auth.log.                                                            |
| R3 | Loss of backup-data integrity or availability                                              | Medium     | Critical | A root path exists on a backup host. Auth.log does not show backup-file access, exfiltration, or tampering.                                                                   |

**Table A.2:** Top source IPs by failed authentication attempts.

| Source IP                  | Failed | Note                                                   |
|----------------------------|--------|--------------------------------------------------------|
| <code>203.0.113.88</code>  | 1,250  | High volume; no accepted login                         |
| <code>198.51.100.88</code> | 1,100  | High volume                                            |
| <code>203.0.113.89</code>  | 951    | High volume + one successful <code>backup</code> login |
| <code>198.51.100.89</code> | 700    | High volume                                            |

**Table A.3:** Most-targeted usernames by failed attempts.

| Username              | Failed | Relevance                                                  |
|-----------------------|--------|------------------------------------------------------------|
| <code>backup</code>   | 1,552  | Sensitive; successful login from <code>203.0.113.89</code> |
| <code>root</code>     | 1,048  | Direct privileged targeting                                |
| <code>admin</code>    | 589    | Common admin target                                        |
| <code>deploy</code>   | 366    | Operational account                                        |
| <code>postgres</code> | 336    | Service/DB account                                         |
| <code>oracle</code>   | 331    | Service/DB account                                         |



**Figure A.1:** Top targeted usernames by failed-password plus maximum-authentication events. `backup` is the most targeted account, supporting its selection as the priority privileged asset.

**Table A.4:** Failed vs accepted logins by time window.

| Window              | Failed | Accepted | Note                                                 |
|---------------------|--------|----------|------------------------------------------------------|
| Night (00:00–05:59) | 3,643  | 149      | Brute-force concentrated here (~70% of all failures) |
| Day (06:00–23:59)   | 1,533  | 1,710    | Legitimate activity dominates business hours         |

Peak failed-login hours are 02:00–04:00; a secondary spike occurs at 23:00. Peak accepted-login hours are 08:00–18:00. The successful **backup** login (Jul 11 03:08:19) falls inside the peak attack window.

These time-window counts are the Appendix A evidence referenced by the main report for the 03:08:19 off-hours finding.

**Table A.5:** Timestamp-order anomalies retained as limitations.

| Line | Previous time          | Current time           | Interpretation                                                |
|------|------------------------|------------------------|---------------------------------------------------------------|
| 13   | 2026-07-06<br>00:49:51 | 2026-07-06<br>00:04:24 | Session-close entry appears after later events in file order. |
| 41   | 2026-07-06<br>02:19:09 | 2026-07-06<br>01:21:46 | Earlier session-close line appears later in the extract.      |
| 70   | 2026-07-06<br>03:40:05 | 2026-07-06<br>02:56:33 | File ordering is not strictly chronological.                  |

## B Raw Log Excerpt (203.0.113.89, Jul 11)

```
Jul 11 03:07:30 backup01 sshd[1675]: Failed password for invalid user oracle from 203.0.113.89
port 56854 ssh2
```

```
Jul 11 03:07:32 backup01 sshd[1447]: Failed password for root from 203.0.113.89 port 40703
ssh2
```

```
Jul 11 03:08:19 backup01 sshd[2164]: Accepted password for backup from 203.0.113.89 port
59595 ssh2
```

```
Jul 11 03:08:19 backup01 sshd[7367]: pam_unix(sshd:session): session opened for user backup
by (uid=0)
```

```
Jul 11 03:08:19 backup01 sudo: backup : TTY=pts/3 ; PWD=/home/backup ; USER=root ;
COMMAND=/usr/bin/journalctl -n 200
```

```
Jul 11 03:08:19 backup01 sudo: backup : TTY=pts/3 ; PWD=/home/backup ; USER=root ;
COMMAND=/usr/bin/systemctl status cron
```

## C Parsing Assumptions and Coverage

- Format assumed: Mon DD HH:MM:SS host process[pid]: message; no year in timestamps.
- Username handling covers both **Failed password for USER** and **Failed password for invalid user USER**.
- Non-standard lines (e.g. **Connection closed by authenticating user ... , reverse mapping ... POSSIBLE BREAK-IN ATTEMPT**) are classified or counted as unmatched, not silently dropped.
- Unmatched-line count: 0.

## D AI-Use Declaration and Workload

- AI-use declaration: Codex GPT-5.5 was used to analyse suspicious points in **4\_auth.log** so they could be brought to the group’s attention for more in-depth analysis. AI was also used to reword and refine the report so that it fits the page limits. Final claims, counts, tables, and figures were checked against **analysis.py** outputs and the assigned log extract.
- Workload record: All members contributed equally.